Bridge Technical Requirements Document

Student talent marketplace | Version 1.1 | 2 November 2026

Purpose

This TRD defines the selected backend technology baseline and technical requirements for Bridge's visible flows: profiles, task listings, proposals, applicant decisions, matches, and text conversations. The selected backend is a separate Python FastAPI service with PostgreSQL persistence. Product rules that the screens do not establish remain open and must be approved before implementation.

System boundary

In scope

- Supabase Auth integration for the visible email/password and Google sign-in paths.
- A separately deployed, versioned HTTPS API service that mediates all marketplace data access.
- Bridge user mapping, user modes, profiles, and skills.
- Task creation, publication/discovery, details, required skills, and ownership.
- Proposal submission, applicant listing, and accept/reject transitions.
- Match/status views and membership-protected text conversation history/send.
- Summary values shown by the UI only after their source and calculation are defined.

Out of scope until approved

Payments/payouts, escrow, ratings algorithms, verification workflows, attachment storage, realtime delivery, recommendations, moderation operations, and administrative tooling.

Selected technology architecture

Concern	Selected technology	Requirement
API service	Python + FastAPI	Own the backend API boundary; expose versioned JSON REST endpoints under `/v1`.
Request validation and schemas	Pydantic	Define request/response models, constraints, and OpenAPI schemas at API boundaries.
Persistence	PostgreSQL	Store transactional marketplace records with foreign keys, unique constraints, and indexes.
ORM/session management	SQLAlchemy 2.x	Use explicit short-lived sessions and transaction boundaries for database work.
Schema migration	Alembic	Version and review schema changes; deploy migrations in a controlled release step.
Identity provider	Supabase Auth	Own sign-in, signup, OAuth, refresh, and signout unless a concrete requirement changes that boundary.
API authorization	FastAPI application layer	Verify token identity, resolve Bridge user, and authorize every object operation.

Request and data path

The mobile client signs in through Supabase Auth, receives an access JWT, then sends that token as a bearer credential to the FastAPI service. FastAPI validates the token, resolves the external subject to a Bridge user, validates the request, checks object-level permissions, calls domain logic, and uses SQLAlchemy to persist records in PostgreSQL. The client does not connect directly to marketplace tables.

Verify JWT signatures using a maintained JWT library and Supabase's published signing keys/JWKS. Check issuer, expiry, and relevant claims. Do not implement cryptographic verification manually or trust a user ID in the request body. Support key rotation according to provider guidance.

Use the least-privileged database role that supports the API's required operations. If database credentials bypass PostgreSQL row-level security, RLS does not protect against application authorization bugs; FastAPI must enforce access rules and authorization tests must cover them. RLS can be added as defense in depth when request identity is safely propagated to the database.

Use one API service for the initial product scope. Do not require microservices, Redis, a task queue, or a separate realtime service until an approved capability or measured operational need requires one.

API requirements

All endpoints use JSON, a version prefix, consistent error envelopes, and ISO-8601 UTC timestamps. Collection endpoints support cursor pagination. Mutating requests are safe to retry where feasible; use idempotency keys for application submissions and decisions if clients may retry after timeouts. FastAPI's OpenAPI schema is the API contract source for client integration.

Method/path or client action	Requirement	Authorization
Supabase Auth SDK: email/password, Google OAuth, refresh, signout	Establish and manage identity session.	Provider-managed; public actions rate-limited.
`GET /v1/me`	Return caller identity and supported user mode(s).	Valid Supabase JWT.
`PATCH /v1/me/profile`	Update allowlisted profile fields.	Authenticated, self only.
`GET /v1/tasks`	Search/browse published tasks with filters and cursor.	Public vs authenticated policy TBD.
`GET /v1/tasks/{id}`	Return public task details.	Public visibility policy TBD.
`POST /v1/tasks`	Create task in agreed initial state.	Task-owner mode.
`PATCH /v1/tasks/{id}`	Edit owned task while allowed by state.	Owner only.
`POST /v1/tasks/{id}/applications`	Submit proposal and portfolio selections.	Student/applicant mode.
`GET /v1/tasks/{id}/applications`	List applicants.	Task owner only.
`PATCH /v1/applications/{id}`	Accept or reject.	Task owner only.
`GET /v1/me/applications`	List caller's applications.	Applicant only.
`GET /v1/me/matches`	List caller's permitted match relationships.	Authenticated.
`GET /v1/conversations/{id}/messages`	Read ordered messages.	Conversation member.
`POST /v1/conversations/{id}/messages`	Append text message.	Conversation member.

Use a stable error shape with machine-readable code, human-readable message, optional field errors, and request identifier. Do not expose stack traces or internal authorization details. Return 401 for invalid/missing credentials and 403 or a privacy-preserving not-found response for inaccessible resources, consistently.

Domain model proposal

Entity	Core attributes and constraints
User	ID, unique Supabase auth user ID, optional normalized email, account status, timestamps.
UserMode	User ID and mode (student, task_owner); cardinality awaits product decision.
Profile	User ID, display name, headline, avatar URL, experience level, weekly availability, portfolio URL. Public response uses explicit allowlist.
Skill	Canonical ID and display name.
ProfileSkill	Profile ID, skill ID, optional ordering; unique pair.
Task	ID, owner ID, title, description, amount, currency, deadline, status, timestamps. Store monetary precision explicitly; do not use floating point.
TaskSkill	Task ID, skill ID; unique pair.
Application	ID, task ID, applicant ID, proposal, state, submitted/updated timestamps. Enforce approved duplicate-application rule.
ApplicationPortfolioItem	Application ID plus item reference or submission-time URL snapshot; policy TBD.
Conversation	ID, optional task/application reference, created timestamp.
ConversationMember	Conversation ID, user ID, joined timestamp; unique pair.
Message	ID, conversation ID, sender ID, body, sent timestamp, optional read timestamp. Append-only recommended.

State and consistency requirements

Proposed task states: draft, published, closed, cancelled. Proposed application states: pending, accepted, rejected, optionally withdrawn. Exact states and transitions must be approved.

- Only an owner can publish/edit/close their task, subject to allowed transitions.
- Only the applicant can create or withdraw their application; only the task owner can decide it.
- The accept operation executes transactionally with task/application updates. If one hire is allowed, prevent concurrent accepts from creating multiple accepted applications.
- Conversation creation and access follow an explicit eligibility rule; recommended: accepted relationship or owner-created invite.
- A caller's `/me` lists never expose another user's private relationships.
- Aggregates such as active-task count and pending payments derive from an authoritative source and have documented semantics.

Validation requirements

- Validate request shape and reject unknown protected fields; enforce maximum text lengths.
- Validate bearer JWT signature and claims; use generic auth failure responses.
- Validate skill IDs against the catalog and limit count/duplicates.
- Enforce valid experience-level enum and non-negative bounded availability.
- Accept only approved URL schemes for portfolio/avatar URLs; never fetch user-provided URLs server-side as validation.
- Validate title, description, budget/currency, deadline, and task state at API and domain boundaries.
- Reject proposals for closed/ineligible tasks; enforce duplicate policy under concurrent requests.

- Validate message size and non-empty body; safely encode at display boundary and enforce membership at every read/write.
- Back critical rules with PostgreSQL constraints and transactions, not request validation alone.

Security and privacy

- Use TLS for all network traffic. Supabase Auth handles password storage and OAuth flows; the Bridge API must not receive or log raw passwords.
- Validate Supabase JWTs on the server and map provider subject to an internal user.
- Apply least privilege and object-level authorization on every task, application, profile, and conversation request.
- Store only necessary personal data. Exclude email and internal identifiers from public profile payloads unless explicitly approved.
- Protect Supabase server secrets and database credentials in a managed secret store; never put service-role keys in the client.
- Do not log bearer tokens, auth credentials, message bodies, or unnecessary personal data.
- Define account deletion, retention, abuse reporting, moderation, and audit event requirements before production.

Reliability, performance, and operations

Numeric SLOs are not in the source and must be set by the owning team. Before launch, define availability, API latency, recovery point/time, data retention, and support hours.

- Use SQLAlchemy/PostgreSQL transactions for state transitions and unique constraints for identity, membership, and duplicate protections.
- Add request IDs, structured logs, and metrics for errors/latency; audit task/application decisions without recording secrets or message content.
- Back up persisted records and verify restoration procedures.
- Paginate task/applicant/message lists and index common filters (status, owner, applicant, task, sent time) after workload is measured.
- Deploy schema migrations through Alembic in a controlled sequence; document backward-compatible rollout and recovery steps.
- Return retry-safe behavior for transient failures without duplicate applications or messages where feasible.

Testing and acceptance

- Contract tests cover success, validation failures, authentication failures, and forbidden access for each private route.
- Authorization tests use multiple users to verify owner/applicant/member boundaries.
- Concurrency tests cover duplicate application submission and competing accept decisions.
- Integration tests cover Supabase token verification, profile save, task publish, browse, apply, review, accept, conversation read/send.
- Verify no private fields appear in public profile/task responses.
- Product acceptance approves state definitions, error messages, pagination, and user-mode rules.

Open technical and product decisions

1. Supabase project configuration, JWT signing-key mode, and deployment/hosting target for FastAPI and PostgreSQL.
2. Public versus authenticated task/profile visibility.
3. User mode cardinality and switching.

4. Currency representation, supported currencies, and whether payments are planned.
5. Task/application cardinalities, state machine, and task closure behavior after acceptance.
6. Chat realtime requirement, notification delivery, read receipts, retention, and conversation eligibility.
7. Rating/verification/earnings source of truth and privacy policy.
8. Required SLOs, expected traffic, data residency, and compliance obligations.

Source

Bridge: Student Talent Marketplace, Google Stitch project 14483777616722284649. Architecture and contracts are proposals; confirm open product decisions before treating them as implementation commitments. The Python/FastAPI stack is the selected technical direction, while hosting details remain undecided.